

Prof. Dr. Peer Küppers

DSCB140
Datenbanken und Datenkunde I

VL7 – Integrität und Transaktionen

Agenda

- **Übersicht Datenintegrität**
- ☐ **Statische Integritätsbedingungen**
- ☐ **Referentielle Integrität**
- ☐ **Ablaufintegrität und Transaktionen**
- ☐ **Zusammenfassung und Ausblick**

Integrität: Definitionen

- Datenbankmanagementsysteme sind für die Erhaltung der **Datenkonsistenz** zuständig
 - Datenkonsistenz: „Die Korrektheit der gespeicherten Daten im Sinn einer widerspruchsfreien und vollständigen Abbildung der relevanten Aspekte des erfassten Realitätsausschnitts.“
Quelle: <https://wirtschaftslexikon.gabler.de/definition/datenintegritaet-28296/version-139767>
- Die Aufgabe eines RDBMS ist die Konsistenzerhaltung insbesondere
 - nach Systemfehlern
 - Recovery-Mechanismen, nicht unser Fokus
 - aufgrund von **Mehrbenutzerbetrieb**
 - Sicherstellung der Ablaufintegrität
 - Umgang mit konkurrierenden Zugriffen auf dieselben Daten
 - zur Sicherstellung **semantischer Integritätsbedingungen**
 - Eigenschaften, die aus dem Datenmodell abgeleitet werden (bspw. Kardinalitäten)
 - Weitere domänenspezifische Eigenschaften, die definiert wurden (bspw. Einschränkungen des Geburtsjahrs)

Integrität: Definitionen

- **Semantische Integritätsbedingungen** (*integrity constraints*):
 - Spezifizieren Bedingungen für die Zulässigkeit oder Korrektheit im Allgemeinen
 - Im Datenbankkontext: Zulässigkeit und Korrektheit in Bezug auf
 - einzelne Datenbankzustände = **statische Integritätsbedingungen**
 - Bedingungen, die zu jedem Zustand der Datenbank erfüllt sein müssen.
 - *Beispiele: nur Datumswerte für Bestelldatum, Matrikelnummer eindeutig, Personalnummer vierstellige Zahl, Professoren haben Rang C2, C3, C4, W2, W3*
 - Zustandsübergänge = **dynamische Integritätsbedingungen**
 - Bedingungen bei Zustandsänderungen in der Datenbank
 - *Beispiele: Der Lagerbestand muss sich nach Auslagerung verringern,*

■ Statische Integritätsbedingungen

Beschränken zulässige Zustände der Datenbank

Diese Arten der Integritätssicherung
haben wir bereits eingesetzt.

- Entitäts- / Schlüsselintegrität
 - Datensätze sind eindeutig identifizierbar
 - Primärschlüssel / UNIQUE-Constraint
- Domänenintegrität / Bereichsintegrität
 - Definition von Datentypen für Attribute
 - Definition von Wertebereichen für Attribute sowie NOT NULL-Constraints
- Referentielle Integrität
 - Definition von Fremdschlüsseln
 - Min-Kardinalitäten durch NOT NULL
- Weitere Benutzerdefinierte Integritätsregeln
 - Beispiel: Zulässige Werte für Kundentypen
 - Die Definition werden wir unten noch kennenlernen.

```
create table kunde_integrity (  
  kdnr int primary key,  
  typ varchar(20) constraint typ_check  
  check (typ in  
         ('Privatkunde', 'Geschäftskunde'))  
)
```

```
create table bestellung_integrity (  
  bestnr int primary key,  
  kunde int not null,  
  foreign key (kunde) references  
  kunde_integrity(kdnr)  
)
```

- **Dynamische Integritätsbedingungen**
Zulässige Zustandsübergänge festlegen
 - Beziehen sich im Gegensatz zu statischen Integritätsbedingungen auf die Änderungen selbst und damit auf mehrere Datenbankzustände.
 - Übergangsbedingungen:
Änderung von altem zu neuem Datenbankzustand wird eingeschränkt
 - Übergang von Familienstand ‚ledig‘ nach ‚geschieden‘ ist nicht zulässig.
 - Gehalt darf nicht kleiner werden.
 - Temporale Bedingungen:
Änderungen in bestimmten zeitlichen Fenstern werden eingeschränkt
 - Gehalt darf innerhalb von 2 Jahren nicht um mehr als 30% wachsen.
 - *Diese Integritätsbedingungen werden wir nicht definieren (möglich bspw. durch Trigger!)*

Semantische Integritätsbedingungen: Reichweite

■ **Reichweite der Bedingung**

- Attributwertbedingungen (bspw. Geburtsjahr > 1900)
- Satzbedingungen (bspw. Geburtsdatum < Einstellungsdatum)
- Satztyp-Bedingungen (bspw. Eindeutigkeit von Attributwerten)
- Satztypübergreifende Bedingungen (bspw. referentielle Integrität zwischen verschiedenen Tabellen)

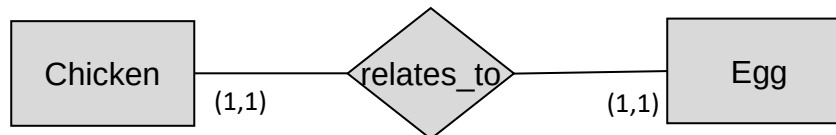
→ Je geringer die Reichweite desto einfacher (=schneller) ist die Überprüfbarkeit

Reichweite

Semantische Integritätsbedingungen: Zeitpunkt der Prüfung

■ Zeitpunkt der Überprüfung

- Unverzögerte Bedingungen
 - Integritätsprüfung nach jeder Datenbank-Zustandsänderung
- Verzögerte Bedingungen
 - Integritätsprüfung nach einer Folge von Datenbank-Zustandsänderungen
 - Benötigt das Transaktionskonzept (s.u.) und verzögerte Bedingungsprüfung
 - Notwendig bei bspw. zyklischen Fremdschlüsselbedingungen:



```
create table chicken (  
    chickenID int primary key,  
    eggID int not null,  
    foreign key (eggID) references egg(eggID)  
)  
create table egg (  
    eggID int primary key,  
    chickenID int not null,  
    foreign key (chickenID) references chicken(chickenID)  
)
```

*Achtung: Die Möglichkeit zur Verzögerung von Integritätsprüfungen bietet MySQL nicht.
Wir müssten temporär Fremdschlüsselprüfungen ausschalten.*

Agenda

- ☐ Übersicht Datenintegrität
- ☒ **Statische Integritätsbedingungen**
- ☐ Referentielle Integrität
- ☐ Ablaufintegrität und Transaktionen
- ☐ Zusammenfassung und Ausblick

Semantische Integritätsbedingungen: Statisch

■ Datenmodell-inhärente, statische Integritätsbedingungen in SQL

- Primärschlüssel: primary key → eindeutig, not null, primärer Identifier
- Schlüsselkandidaten: unique → eindeutig
- NULL-Werte verbieten: not null
- Standardwerte angeben (wenn NULL erlaubt): default

```
create table studierende (  
    matrikelnr int primary key,      -- Primärschlüssel  
    name varchar(30) not null,      -- Name ist anzugeben  
    email varchar(50) unique,       -- Emailadresse muss eindeutig sein  
    semester int default 1          -- Standardwert  
)
```

- Fremdschlüssel: foreign key → Einstellmöglichkeiten für referentielle Integrität (s.u.)

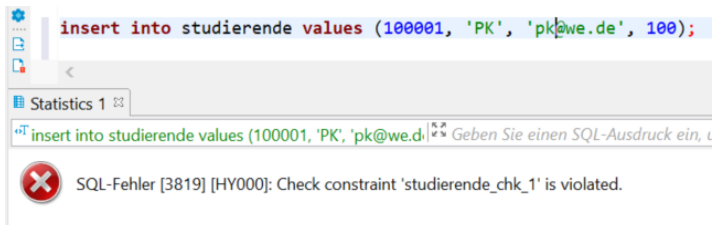
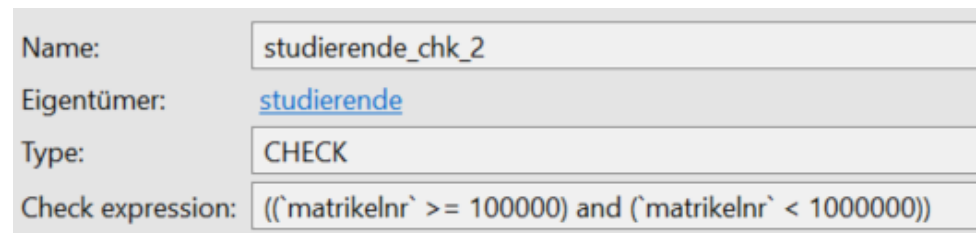
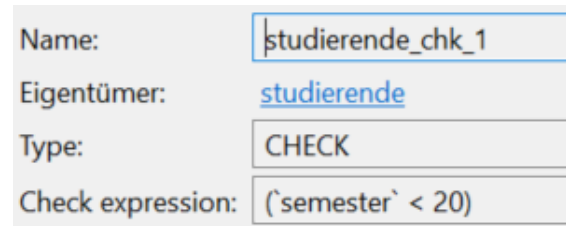
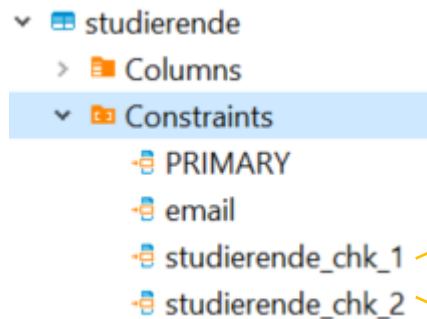
Semantische Integritätsbedingungen: Statisch

- Spezielle, statische Integritätsbedingungen in SQL
 - Definierbar durch das CHECK-Statement
 - Spezifikation allgemeiner, statischer Integritätsbedingung
 - Auswertung der Check-Klausel bei Änderung/Einfügen von Datensätzen in die Tabelle
 - Änderung/Einfügen wird nur zugelassen, wenn die Bedingung erfüllt ist
 - Bedingungen können je Spalte im CHECK-Statement definiert werden
- ```
create table studierende (
 matrikelnr int primary key, -- Primärschlüssel
 name varchar(30) not null, -- Name ist anzugeben
 email varchar(50) unique, -- Emailadresse muss eindeutig sein
 semester int default 1 -- Standardwert
 check (semester < 20) -- domänenspezifische Prüfung
)
```
- Komplexe, zusammengesetzte Bedingungen können formuliert werden (AND, OR, NOT, ...)

# Semantische Integritätsbedingungen: Statisch

- Beispiele für statische Integritätsbedingungen in SQL
  - Durch ALTER TABLE können nachträglich CHECK-Statements hinzugefügt werden

```
-- Matrikelnummer 6-stellig und beginnend bei 100000
alter table studierende
add check (matrikelnr >=100000 and matrikelnr < 1000000);
```



# Semantische Integritätsbedingungen: Statisch

## ■ Beispiele für statische Integritätsbedingungen in SQL

- Mittels der IN-Prüfung können konstante Werte festgelegt werden:

```
-- Feld "status" hinzufügen und mögliche Eingaben beschränken
alter table studierende
add column status varchar(20) check (status in ('aktiv', 'urlaubssemester'));
```

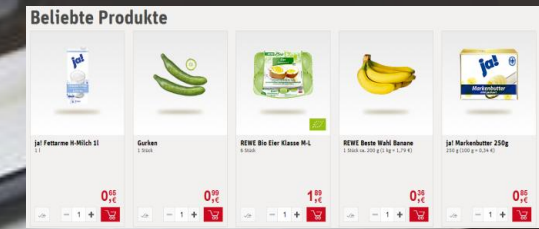
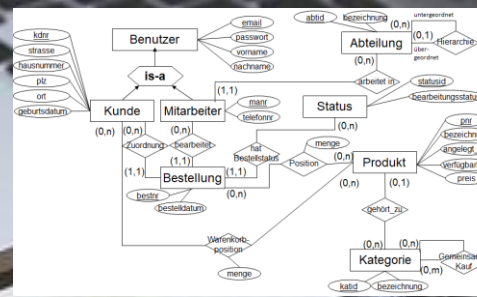
- Manche RDBMS ermögliche die Definition von „Domains“ für diese Anforderung
  - Domänenspezifischer Datentyp mit Möglichkeit, CHECK zu definieren
  - *In MySQL nicht möglich, PostgreSQL und andere erlauben dies!*

```
-- Alternativ kann ein "eigener Datentyp" inklusive CHECK definiert werden
create domain studierenden_status as varchar(20)
default 'aktiv'
check (value in ('aktiv', 'urlaubssemester'));
```

```
alter table studierende
add column status studierenden_status; -- Verwendung der DOMAIN
```

- Manche RDBMS erlauben Subqueries im CHECK-Statement (*MySQL nicht*)

# Fallstudie



- IUT 1.1 ■ Fügen Sie mittels ALTER TABLE ein CHECK-Constraint hinzu, so dass in der Tabelle Produkt die Verfügbarkeit nicht kleiner 0 eingestellt werden kann.
- IUT 1.2 ■ Versuchen Sie mittels UPDATE, alle Verfügbarkeiten zu invertieren (also \* (-1)).
- IUT 1.3 ■ Entfernen Sie die Prüfung (Verfügbarkeit  $\geq 0$ ) wieder mittels ALTER TABLE. Dazu müssen Sie den Namen des Constraints zunächst suchen (bspw. in DBeaver).
- IUT 1.4 ■ Fügen Sie erneute die Prüfung aus IUT 1.1 hinzu. Geben Sie zusätzlich als obere Grenze der Verfügbarkeit 100000 an.
- IUT 1.5 ■ Testen Sie, ob der letzte CHECK aus IUT 1.4 funktioniert (Update einzelner Produkte).

# Agenda

- ☐ Übersicht Datenintegrität
- ☐ Statische Integritätsbedingungen
- ☒ Referentielle Integrität
- ☐ Ablaufintegrität und Transaktionen
- ☐ Zusammenfassung und Ausblick

# Referentielle Integrität – Grundlagen

## ■ Referentielle Integrität

- Definition referentielle Integrität:  
Fremdschlüssel müssen entweder auf existierende Tupel einer anderen Relation verweisen oder einen Nullwert enthalten.
  - Achtung: Abhängig von der Kardinalität kann NOT NULL definiert sein!
- Wenn auf ein nicht existierendes Tupel verwiesen wird spricht man von einer **„dangling reference“**
  - Das Auftreten von „dangling references“ soll durch das RDBMS verhindert werden!
- Um „dangling references“ zu verhindern muss sowohl beim Einfügen, als auch insbesondere beim Löschen und Aktualisieren von Daten auf die Gültigkeit der Referenzen geachtet werden.
  - Die Überprüfung und das Handling von UPDATES / DELETES kann gesteuert werden durch unterschiedliche „Überprüfungsmodi“.



# Referentielle Integrität – Referenzoptionen

## ■ Referentielle Integrität – Referenzoptionen

- Beim Anlegen eines Fremdschlüssels können Einstellungen vorgegeben werden, was beim Aktualisieren oder Löschen von Datensätzen passieren soll.
  - Hinweis: Beim Einfügen von Daten wird die Gültigkeit der Referenz ohnehin geprüft.
  - Der Parameter steuert, wie sich die Datenbank beim Löschen/Aktualisieren verhalten soll.

|   |                                                         |                                                                                                                     |
|---|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| 1 | [CONSTRAINT <i>[symbol]</i> ] FOREIGN KEY               |                                                                                                                     |
| 2 | [ <i>index_name</i> ] ( <i>col_name</i> , ...)          |                                                                                                                     |
| 3 | REFERENCES <i>tbl_name</i> ( <i>col_name</i> , ...)     |                                                                                                                     |
| 4 | [ON DELETE <i>reference_option</i> ]                    | Für beide Event-Type (Löschen und Aktualisieren) kann ein Parameter (die <i>reference_option</i> ) vergeben werden. |
| 5 | [ON UPDATE <i>reference_option</i> ]                    |                                                                                                                     |
| 6 |                                                         |                                                                                                                     |
| 7 | <i>reference_option</i> :                               | Durch diese Optionen wird die Konsistenz auf verschiedene Arten sichergestellt.                                     |
| 8 | RESTRICT   CASCADE   SET NULL   NO ACTION   SET DEFAULT |                                                                                                                     |

Werden die Optionen nicht spezifiziert ist dies der Standard!

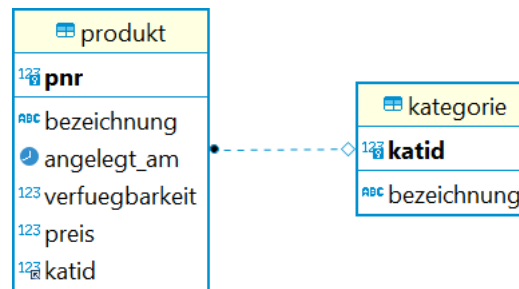
In MySQL nicht verfügbar!

- RESTRICT: Aktualisieren oder Löschen ablehnen falls dadurch „dangling references“ in Tabellen entstehen, die auf den zu löschenden/aktualisierenden Datensatz zeigen.
- SET NULL: Referenzen auf NULL setzen, falls „dangling reference“ entsteht.
- SET DEFAULT: Referenzen auf Standardwert setzen, falls „dangling reference“ entsteht.
- NO ACTION: äquivalent zu RESTRICT
- CASCADE: s.u.

Quelle: <https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html>

# Referentielle Integrität – Referenzoptionen

- Referentielle Integrität – Referenzoption „ON DELETE CASCADE“
  - Entstehen beim Löschen eines Datensatzes aus einer Tabelle A „dangling references“ in einer auf diesen Datensatz verweisenden Tabelle B, dann werden in Tabelle B alle Datensätze mit „dangling reference“ automatisch gelöscht.
  - Beispiel:



| pnr | bezeichnung            | angelegt_am         | verfuegbarkeit | preis | katid  |
|-----|------------------------|---------------------|----------------|-------|--------|
| 1   | Schokoladenpudding     | 2020-11-24 12:15:59 | 100            | 0,8   | 1      |
| 2   | Eier (Freilandhaltung) | 2020-11-24 13:16:59 | 300            | 2,2   | 2      |
| 3   | Banane                 | 2020-11-24 14:10:55 | 150            | 0,23  | 4      |
| 4   | Zitrone                | 2020-11-25 11:13:59 | 80             | 0,34  | 4      |
| 5   | Blumenkohl             | 2020-10-24 12:15:59 | 10             | 1,8   | 5      |
| 6   | Grillwurst             | 2020-10-24 12:14:48 | 50             | 3,6   | 6      |
| 7   | Weißer Schokolade      | 2020-12-01 09:15:59 | 90             | 1,1   | 3      |
| 8   | Zartbitterschokolade   | 2020-11-28 16:15:59 | 106            | 0,9   | 3      |
| 9   | Vollmilch              | 2020-09-24 13:13:23 | 80             | 0,9   | 1      |
| 10  | Katzenfutter           | 2020-11-30 12:17:42 | 10             | 2,2   | [NULL] |
| 11  | Hundefutter            | 2020-11-30 12:17:48 | 18             | 2,4   | [NULL] |

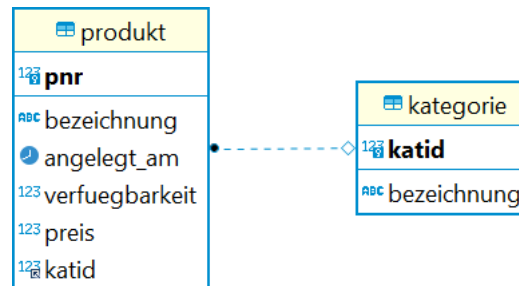
Kaskadierendes  
Löschen

| katid | bezeichnung      |
|-------|------------------|
| 1     | Molkereiprodukte |
| 2     | Eier             |
| 3     | Süßwaren         |
| 4     | Obst             |
| 5     | Gemüse           |
| 6     | Fleisch          |
| 7     | Fisch            |

**delete from kategorie  
where katid=1;**

# Referentielle Integrität – Referenzoptionen

- Referentielle Integrität – Referenzoption „ON UPDATE CASCADE“
  - Entstehen beim Ändern der Werte eines Attributs in einer Tabelle A, auf das mittels Fremdschlüssel aus einer Tabelle B verwiesen wird, in Tabelle B „dangling references“, dann werden in Tabelle B alle Datensätze mit „dangling reference“ automatisch auf den neuen Wert des Attributs gesetzt.
  - Beispiel:



| pnr | bezeichnung            | angelegt_am         | verfuegbarkeit | preis | katid  |
|-----|------------------------|---------------------|----------------|-------|--------|
| 1   | Schokoladenpudding     | 2020-11-24 12:15:59 | 100            | 0,8   | 1      |
| 2   | Eier (Freilandhaltung) | 2020-11-24 13:16:59 | 300            | 2,2   | 10     |
| 3   | Banane                 | 2020-11-24 14:10:55 | 150            | 0,23  | 4      |
| 4   | Zitrone                | 2020-11-25 11:13:59 | 80             | 0,34  | 4      |
| 5   | Blumenkohl             | 2020-10-24 12:15:59 | 10             | 1,8   | 5      |
| 6   | Grillwurst             | 2020-10-24 12:14:48 | 50             | 3,6   | 6      |
| 7   | Weißer Schokolade      | 2020-12-01 09:15:59 | 90             | 1,1   | 3      |
| 8   | Zartbitterschokolade   | 2020-11-28 16:15:59 | 106            | 0,9   | 3      |
| 9   | Vollmilch              | 2020-09-24 13:13:23 | 80             | 0,9   | 1      |
| 10  | Katzenfutter           | 2020-11-30 12:17:42 | 10             | 2,2   | [NULL] |
| 11  | Hundefutter            | 2020-11-30 12:17:48 | 18             | 2,4   | [NULL] |

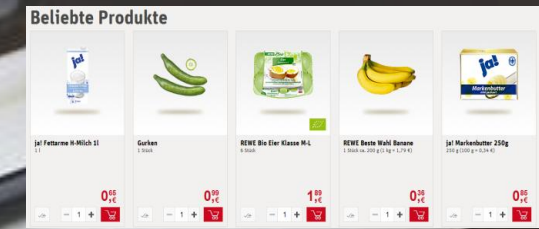
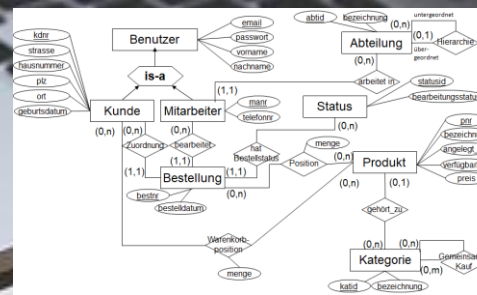
Kaskadierendes  
Aktualisieren

| katid | bezeichnung      |
|-------|------------------|
| 1     | Molkereiprodukte |
| 10    | Eier             |
| 3     | Süßwaren         |
| 4     | Obst             |
| 5     | Gemüse           |
| 6     | Fleisch          |
| 7     | Fisch            |

**update kategorie**  
**set katid=10**  
**where katid=2;**

**Achtung:** Bei vielen Tabellen, die mit Fremdschlüsseln verknüpft und CASCADE eingestellt ist, dass das Löschen eines Datensatzes in der Datenbank viel bewirken (viele Datensätze können als Konsequenz automatisch gelöscht werden).

# Fallstudie



- IUT 2.1 ■ Finden Sie heraus, auf welche Optionen für ON DELETE und ON UPDATE der Fremdschlüssel von der Tabelle Produkt auf die Tabelle Kategorie (katid) eingestellt ist.
- Ansatz 1: durch Ausprobieren (Löschen und Aktualisieren von Datensätzen)
  - Ansatz 2: durch Prüfen der „Foreign Key“-Parameter in DBeaver
- IUT 2.2 ■ Stellen Sie für den Fremdschlüssel „katid“ in Produkt ein, dass beim Löschen die Kategorie auf NULL gesetzt wird und beim Aktualisieren kaskadierend der Wert weitergegeben wird.
- Achtung: Dazu müssen Sie zuerst das Foreign Key-Constraint komplett löschen und dann neu den Fremdschlüssel definieren.
    - Löschen des Foreign Key-Constraints: `alter table produkt drop constraint produkt_ibfk_1;`
    - Neu Anlegen des Foreign Keys: `alter table produkt add constraint produkt_ibfk_1 foreign key (katid) references kategorie(katid) on delete set null on update cascade;`
  - Testen Sie die Einstellungen (vgl. Aufgabe IUT 2.1)

# Agenda

- ☐ Übersicht Datenintegrität
- ☐ Statische Integritätsbedingungen
- ☐ Referentielle Integrität
- ☒ Ablaufintegrität und Transaktionen
- ☐ Zusammenfassung und Ausblick

# Ablaufintegrität – Einführendes Beispiel

- Beispieltabelle „Konto“:

| <u>KtoNr</u> | Saldo |
|--------------|-------|
| 1            | 100   |
| 2            | 100   |

- Beispielaufgabe 1

- Wir wollen eine Überweisung von 50€ Konto 1 auf Konto 2 durchführen.

1 **update** Konto **set** Saldo = Saldo - 50 **where** KtoNr = 1;

2 **update** Konto **set** Saldo = Saldo + 50 **where** KtoNr = 2;

Client-Absturz nach (1)

- Problem

- Was passiert, wenn unser Rechner (Datenbankclient) nach dem ersten Update abstürzt?
- Überweisung kommt nicht an, ist auf dem Konto aber schon abgebogen.
- Fehlerhafte Daten, Inkonsistenz!

# Ablaufintegrität – Einführendes Beispiel

- Beispieltabelle „Konto“:

| <u>KtoNr</u> | Saldo |
|--------------|-------|
| 1            | 100   |
| 2            | 100   |

- Beispielaufgabe 2

- Wir wollen eine Überweisung von 80€ von Konto 1 auf Konto 2 durchführen.
- Diese soll aber nur möglich sein, wenn Konto 1 nicht überzogen wird ( $\text{Saldo} \geq 0$ )

1 Prüfung, ob Saldo ausreicht.



Eine zweite Überweisung wird  
nach (1) gestartet. Saldoprüfung vor (2)

2 **update** Konto **set** Saldo = Saldo - 60 **where** KtoNr = 1;

3 **update** Konto **set** Saldo = Saldo + 60 **where** KtoNr = 2;

- Problem

- Was passiert, wenn nach der Saldoprüfung (1) direkt eine andere Überweisung gestartet wird? Die zweite Saldoprüfung erfolgt also vor dem Update (2).
  - Das Konto könnte überzogen werden.
  - Fehlerhafte Daten, Inkonsistenz!

# Ablaufintegrität – Einführendes Beispiel

## ■ Konsequenz

- Wir benötigen einen Mechanismus, der mehrere Operationen zu einer logischen Einheit zusammenfassen kann. Diese soll bei Fehlern alle vorherigen Operationen rückgängig machen können (Beispielaufgabe 1) oder bei Bedarf andere Operationen blockieren können (Beispielaufgabe 2).

## ■ Ansatz in RDBMS: Transaktionen

- Eine Transaktion ist eine Folge von Operationen (Einfügen, Löschen, Ändern und/oder Selektieren von Daten), die die Datenbank von einem konsistenten Zustand in einen konsistenten, eventuell veränderten, Zustand überführt.
- Wichtig
  - Semantische Integrität: Korrekter (konsistenter) DB-Zustand nach Ende der Transaktion
  - Ablaufintegrität: Fehler durch gleichzeitigen, konkurrierenden Zugriff mehrerer Benutzer auf dieselben Daten vermeiden

→ Transaktionen müssen also gewissen Anforderungen entsprechen.

→ Diese werden das ACID-Prinzip genannt (s. nächste Folie).



# ACID-Eigenschaft von RDBMS und Transaktionen

## ■ ACID (1/2)

|          |                                       |
|----------|---------------------------------------|
| <b>A</b> | Atomarität ( <i>atomicity</i> )       |
| <b>C</b> | Konsistenz ( <i>consistency</i> )     |
| <b>I</b> | Isolation ( <i>isolation</i> )        |
| <b>D</b> | Dauerhaftigkeit ( <i>durability</i> ) |

## ■ Atomicity (Atomarität)

*„Alles oder nichts“*

- Die Teilschritte einer Transaktion werden vom DBMS als eine **unteilbare, atomare Einheit** durchgeführt („alle oder keiner“).

## ■ Consistency (Konsistenz)

*„Daten bleiben konsistent“*

- Die Datenbank hat vor und nach einer Transaktion stets einen konsistenten Zustand, d.h. alle Integritätsbedingungen des Datenbankschemas sind erfüllt.

# ACID-Eigenschaft von RDBMS und Transaktionen

## ■ ACID (2/2)

|          |                                       |
|----------|---------------------------------------|
| <b>A</b> | Atomarität ( <i>atomicity</i> )       |
| <b>C</b> | Konsistenz ( <i>consistency</i> )     |
| <b>I</b> | Isolation ( <i>isolation</i> )        |
| <b>D</b> | Dauerhaftigkeit ( <i>durability</i> ) |

## ■ Isolation

- Eine Transaktion läuft isoliert gegenüber dem Einfluss anderer Transaktionen ab, so als ob sie exklusiven Zugriff auf die Daten hätte.
- Eventuell wird die Transaktion vom DBMS abgebrochen, weil andernfalls ein unerlaubter Einfluss anderer Transaktionen erfolgt wäre.

*„Eine Transaktion  
hat die Daten  
quasi für sich  
allein“*

## ■ Durability

- Die Ergebnisse einer bestätigten Transaktion sind dauerhaft gesichert, d.h. das DBMS garantiert, dass bei einem Fehler der bestätigte Zustand wiederhergestellt werden kann.

*„Nichts  
geht  
verloren“*

# Abbruch einer Transaktion

- Um die ACID-Eigenschaften zu erfüllen müssen Transaktionen wenn nötig abgebrochen werden.
- Transaktionen werden **abgebrochen** durch
  - Manuellen Eingriff des Benutzers
    - Wenn der Nutzer während der Transaktion interaktiven Zugriff hat (bspw. in DBeaver)
  - Ausnahmebedingungen und Programmierfehler
    - Prüfungen innerhalb der Transaktion, die zu einem Abbruch führen
    - Fehler im SQL-Code, Division durch Null, etc.
  - Nebenläufigkeitskontrolle bei Mehrbenutzerbetrieb
    - Wenn das DBMS eine Verklemmung oder Verletzung der Serialisierbarkeit entdeckt.
      - Serialisierbarkeit: die Ausführung mehrerer paralleler Transaktionen führt zum selben Ergebnis wie eine serielle Ausführung („nacheinander“) der Transaktionen.
      - Kann dies nicht gewährleistet werden, können Inkonsistenzen entstehen.
  - Schwere Fehler (Systemabsturz, Hardwareausfall) → Absturz DBMS
    - Auch in dem Fall werden laufende Transaktionen unterbrochen. Diese müssen in den Logging- und Recovery-Mechanismen der Datenbank berücksichtigt werden!

# Abbruch einer Transaktion

- Handling abgebrochener Transaktionen
  - Beim Abbruch einer Transaktion werden mittels „rollback“ Zwischenstände der Operationen rückgängig gemacht. → Konsistenz herstellen
- Bricht die Transaktion nicht ab, dann werden mittels „commit“ die Ergebnisse der Transaktion persistiert.
  - Die Transaktion wird als gültig erklärt.
- Vgl. **Schritte 6 und 7** der folgenden Übung

# Übung “Transaktionen in RDBMS”



- Übung (<https://pkuep.github.io/hska-140-db>)
  - Wir möchten den Sinn von Transaktionen zunächst anhand eines konkreten Beispiels kennenlernen.

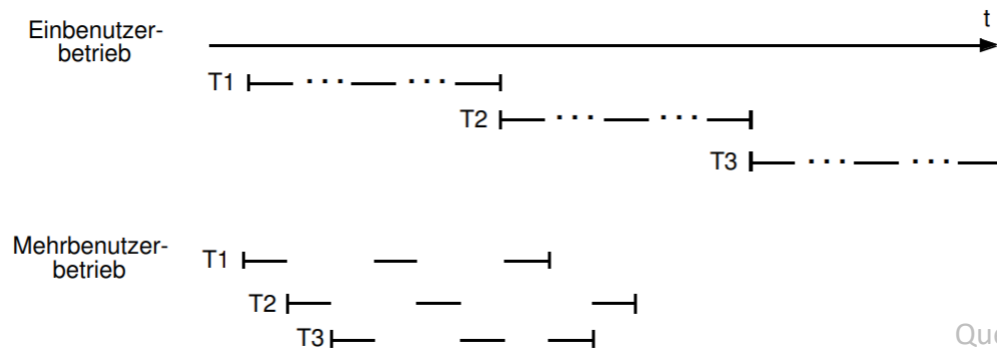
## Transaktionen in relationalen Datenbanken



Zweck und Handling von Transaktionen (Fallstudie Webshop)

# Isolation von Transaktionen

- Durch den Mehrbenutzerbetrieb können sogenannte Anomalien auftreten (s.u.).
  - Transaktionen werden im Mehrbenutzerbetrieb nebenläufig ausgeführt.



Quelle: <https://vsis-www.informatik.uni-hamburg.de/>

- Das DBMS muss somit für Isolation sorgen.
  - „Logischer Einbenutzerbetrieb“ ist das Ziel.
  - Im Beispiel haben wir gesehen, dass die Transaktionen nicht vollständig isoliert voneinander liefen (wir konnten einen Zwischenstand der Transaktion „sehen“).
    - Je nach Grad der zu vermeidenden Anomalien muss ggf. mit langen Antwortzeiten und Blockaden gerechnet werden. → Problematisch bei großen Nutzerzahlen!
    - Es gibt daher verschiedene Einstellungen für Isolation.

# Anomalien im unkontrollierten Mehrbenutzerbetrieb

---

Anomalieklassen im **unkontrollierten** Mehrbenutzerbetrieb:

- **Lost Update**

- Verloren gegangene Änderungen

- **Dirty Read**

- Zugriff auf „schmutzige“ Daten (inkonsistente Zwischenstände der Datenbank)
- *Dies haben wir im Beispiel gesehen (eine neue Bestellung existierte kurzzeitig)*

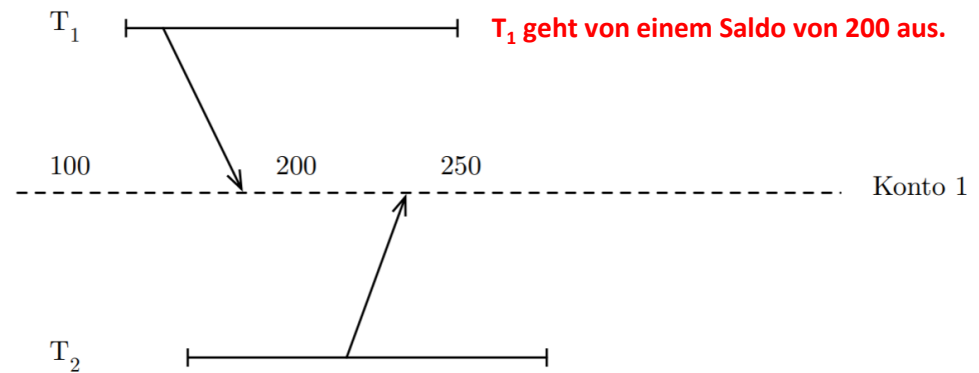
- **Non-repeatable Read**

- Nicht-reproduzierbares Lesen, d.h. inkonsistente Analysen der Datenbank
- *Beispiel: bspw. dieselbe Zeile wird in einer Transaktion gelesen und enthält unterschiedliche Werte*

- **Phantom Read**

- Unterschiedliche Zeilen werden von identischen Abfragen innerhalb einer Transaktion zurückgeliefert

# Lost Update



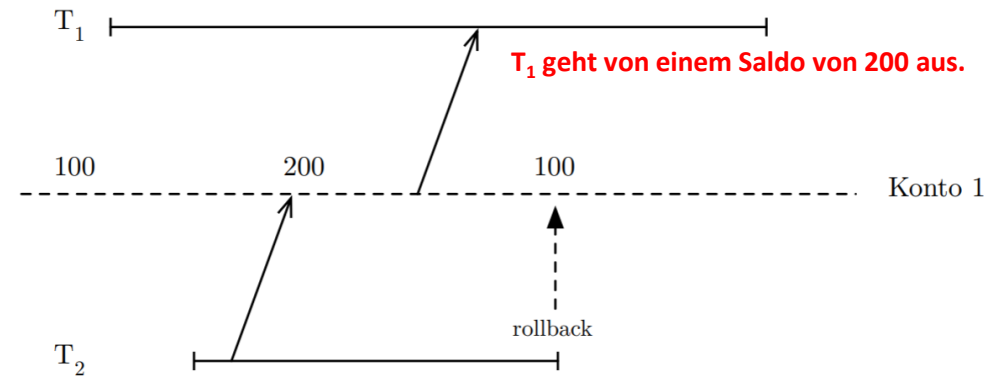
## ■ Lost Update

- Die Änderung von Transaktion  $T_2$  hat die Änderung von Transaktion  $T_1$  überschrieben.

| $T_1$                                              | $T_2$                                              | Saldo |
|----------------------------------------------------|----------------------------------------------------|-------|
|                                                    |                                                    | 100   |
| update Konto<br>set Saldo = 200<br>where KtoNr = 1 |                                                    | 200   |
|                                                    | update Konto<br>set Saldo = 250<br>where KtoNr = 1 | 250   |
| commit                                             |                                                    | 200   |
|                                                    | commit                                             | 250   |



# Dirty Read



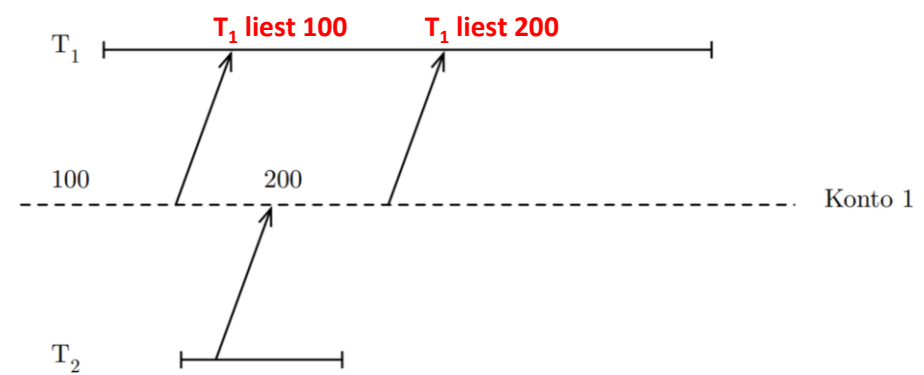
## ■ Dirty Read

- Transaktion  $T_1$  hat einen Wert verwendet, der niemals gültig war!

| $T_1$                                                                                | $T_2$                                              | Saldo für Konto 1 |
|--------------------------------------------------------------------------------------|----------------------------------------------------|-------------------|
|                                                                                      |                                                    | 100               |
|                                                                                      | update Konto<br>set Saldo = 200<br>where KtoNr = 1 | 200               |
| select Saldo<br>from Konto<br>where KtoNr = 1<br>...<br>$T_1$ verwendet den Wert 200 |                                                    |                   |
|                                                                                      | rollback                                           | 100               |
|                                                                                      |                                                    | 100               |
| commit                                                                               |                                                    |                   |

Quelle: <https://esb-dev.github.io/mat/dbs-trans-bh.pdf>

# Non-repeatable Read



## ■ Non-repeatable Read

- $T_1$  kann sich nicht darauf verlassen, dass ein gelesener Wert gültig bleibt!

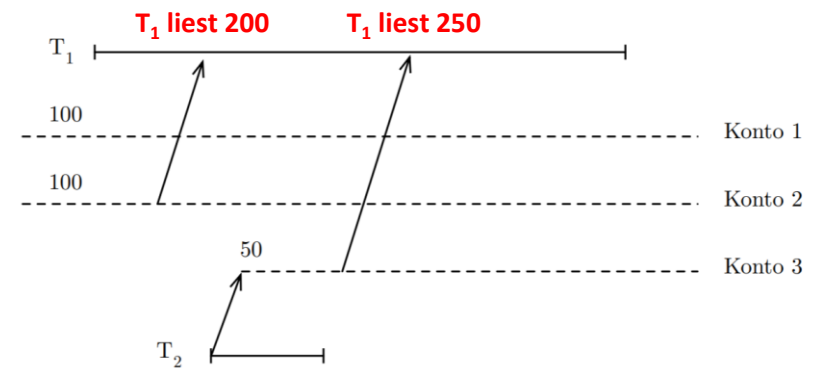
| $T_1$                                                                                  | $T_2$                                                              | Saldo für Konto 1 |
|----------------------------------------------------------------------------------------|--------------------------------------------------------------------|-------------------|
|                                                                                        |                                                                    | 100               |
| <pre>select Saldo from Konto where KtoNr = 1 <math>T_1</math> liest den Wert 100</pre> |                                                                    |                   |
|                                                                                        | <pre>update Konto set Saldo = 200 where KtoNr = 1 ... commit</pre> |                   |
|                                                                                        |                                                                    | 200               |
| <pre>select Saldo from Konto where KtoNr = 1 <math>T_1</math> liest den Wert 200</pre> |                                                                    |                   |

# Non-repeatable Read

## ■ Non-repeatable Read: Beispielfehler

| $T_1$                                                                | $T_2$                                                                  | Saldo Konten 1-3         |
|----------------------------------------------------------------------|------------------------------------------------------------------------|--------------------------|
|                                                                      |                                                                        | 40, 50, 30<br>Summe: 120 |
| <hr/>                                                                |                                                                        |                          |
| $T_1$ liest Saldo von Konto 1 und schreibt den Wert in sum, also 40. |                                                                        |                          |
| <hr/>                                                                |                                                                        |                          |
|                                                                      | $T_2$ ändert Saldo von Konto 3 auf 20 und von Konto 1 auf 60<br>commit | 60, 50, 20<br>Summe: 130 |
| <hr/>                                                                |                                                                        |                          |
| $T_1$ liest Saldo von Konto 2 und addiert den Wert zu sum, also 90.  |                                                                        |                          |
| <hr/>                                                                |                                                                        |                          |
| $T_1$ liest Saldo von Konto 3 und addiert den Wert zu sum, also 110. |                                                                        |                          |
| <hr/>                                                                |                                                                        |                          |

# Phantom Row



## ■ Phantom Row

- $T_1$  bekommt Datensätze „untergeschoben“ (ähnlich zu non-repeatable read).

| $T_1$                                                                                 | $T_2$                                                                               | Saldo für Konten |
|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|------------------|
|                                                                                       |                                                                                     | 100, 100         |
| <code>select sum(Saldo)</code><br><code>from Konto</code><br>$T_1$ liest den Wert 200 |                                                                                     |                  |
|                                                                                       | <code>insert into Konto</code><br><code>values (3,50)</code><br><code>commit</code> | 100, 100, 50     |
| <code>select sum(Saldo)</code><br><code>from Konto</code><br>$T_1$ liest den Wert 250 |                                                                                     |                  |

# Vermeidung von Anomalien durch DBMS

## ■ Definition von Isolationsleveln in SQL

- Je weniger Anomalien unterbunden werden sollen, desto aufwändiger ist das Transaktionshandling und desto geringer kann die Lese-/Schreibperformance werden.

| Isolationslevel         | Anomalie-<br>klasse | Lost Update /<br>Dirty Read | Non-Repeatable<br>Read | Phantom Row   |
|-------------------------|---------------------|-----------------------------|------------------------|---------------|
|                         |                     |                             |                        |               |
| <b>READ UNCOMMITTED</b> |                     | Möglich                     | Möglich                | Möglich       |
| <b>READ COMMITTED</b>   |                     | Nicht möglich               | Möglich                | Möglich       |
| <b>REPEATABLE READ</b>  |                     | Nicht möglich               | Nicht möglich          | Möglich       |
| <b>SERIALIZABLE</b>     |                     | Nicht möglich               | Nicht möglich          | Nicht möglich |

“[...] you can relax the consistency rules with READ COMMITTED or even READ UNCOMMITTED, in situations such as bulk reporting where precise consistency and repeatable results are less important than minimizing the amount of overhead for locking.”

(MySQL Manual, 15.7.2.1 Transaction Isolation Levels)

# Vermeidung von Anomalien durch DBMS

## ■ Definition von Isolationsleveln in MySQL

- Mittels folgender Befehle kann in MySQL der Isolationsgrad gesetzt werden:

**SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED**  
**READ COMMITTED**  
**REPEATABLE READ**  
**SERIALIZABLE**

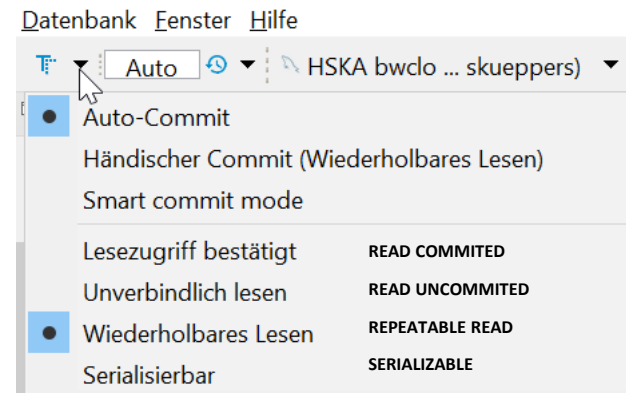
Isolationsstärke

Vgl. <https://dev.mysql.com/doc/refman/8.0/en/set-transaction.html>

- Dies bezieht sich auf die gestartete Transaktion (oder in DBeaver auf die gesamte interaktive Datenbanksession).

- In DBeaver kann der Isolationsgrad der Session auch per GUI eingestellt werden.

- Achtung: andere Reihenfolge



# Vermeidung von Anomalien durch DBMS

| KtoNr | Saldo |
|-------|-------|
| 1     | 100   |
| 2     | 100   |

## ■ Übung

- Erzeugen Sie in einer DBeaver Session **A** die Tabelle „Konto“ und füllen Sie sie:

```
create table Konto (KtoNr int primary key, Saldo float);
insert into Konto values (1, 100), (2, 100);
```

- Stellen Sie nun Session A auf manuelles COMMIT um:
- Aktualisieren Sie eine Zeile, z.B.:

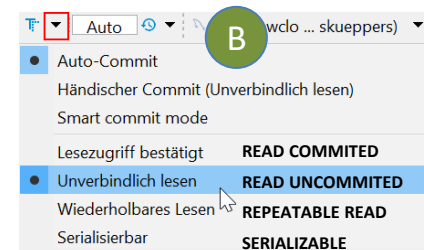
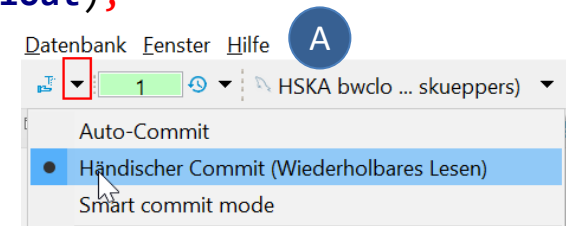
```
update Konto set Saldo=Saldo+10 where KtoNr=1;
```

- Achtung: es wurde eine Transaktion gestartet, aber noch nicht abgeschlossen.
- DBeaver zeigt dies an: 
- Wir belassen es nun bei diesem Zustand in Session A. Commit/Rollback nutzen wir später.

- Legen Sie eine weitere Session **B** (SQL-Script) und stellen Sie ihr Transaktions-Isolation-Level auf „READ UNCOMMITTED“:
- Lesen Sie die gesamte Tabelle Konto aus:

```
select * from Konto;
```

- Analysieren Sie den Effekt der verschiedenen Isolationslevel.
  - Probieren Sie dabei auch in Session A die Möglichkeit aus, die Transaktion (also das UPDATE) zu bestätigen (COMMIT) oder rückgängig zu machen (ROLLBACK).



# Übung “Transaktionen in RDBMS”



- Übung (<https://pkuep.github.io/hska-140-db>)
  - Wir möchten den Sinn von Transaktionen zunächst anhand eines konkreten Beispiels kennenlernen.

## Transaktionen in relationalen Datenbanken



Zweck und Handling von Transaktionen (Fallstudie Webshop)



# Agenda

- ☐ Übersicht Datenintegrität
- ☐ Statische Integritätsbedingungen
- ☐ Referentielle Integrität
- ☐ Ablaufintegrität und Transaktionen
- ☒ Zusammenfassung und Ausblick

# Zusammenfassung und Ausblick

---



- Sie haben nun einen detaillierteren Einblick in das Thema Integrität erhalten und können unterschiedliche Arten der Integrität voneinander abgrenzen.
- Sie können statische Integritätsbedingungen in SQL formulieren und kennen die unterschiedlichen Parametrisierungen der referentiellen Integrität sowie deren Auswirkungen auf Aktualisierungs- bzw. Löschoperationen.
- Sie haben die Notwendigkeit von Transaktionen verstanden und wissen, wie Transaktionen definiert werden. Sie kennen die üblicherweise unterschiedenen Anomalien in Datenbanken und Möglichkeiten, diese durch verschiedene Isolationsgrade von Transaktionen zu verhindern.

**Vielen Dank für Ihre Aufmerksamkeit!**

